



Neural Information Processing in Hierarchical Prototypical Networks

Zilong Ji¹, Xiaolong Zou², Xiao Liu¹, Tiejun Huang², Yuanyuan Mi³,
and Si Wu²(✉)

¹ State Key Laboratory of Cognitive Neuroscience and Learning,
Beijing Normal University, Beijing 100875, China

² School of Electronics Engineering and Computer Science, IDG/McGovern Institute
for Brain Research, Peking University, Beijing 100871, China
siwu@pku.edu.cn

³ Institute for Neurointelligence, School of Medicine, Chongqing University,
Chongqing, China

Abstract. Prototypical networks (PTNs), which classify unseen data points according to their distances to the prototypes of classes, are a promising model to solve the few-shot learning problem. Mimicking the characteristics of neural systems, the present study extends PTNs in two aspects. Firstly, we develop hierarchical prototypical networks (HPTNs), which construct prototypes at all layers and minimize the weighted classification errors of all layers. Applied to two benchmark datasets, we show that a HPTN has comparable, or slightly better, performances than a PTN. We also find that after training, the HPTN generates good prototype representations in the intermediate layers of the network. Secondly, we demonstrate that the classification operation via distance computation in a PTN can be replaced approximately by the attracting dynamics of the Hopfield model, indicating the potential realization of metric-learning in neural systems. We hope this study establishes a link between PTNs and neural information processing.

Keywords: Prototype · Few-shot learning · Metric-learning
Hopfield model

1 Introduction

Recently, a deep neural network model, called prototypical networks (PTNs), was proposed to solve the few-shot learning problem [1]. The formulation of a PTN is based on metric-learning, as it classifies unseen data points according to their distances to the prototypes of different classes. Moreover, these prototypes, which are constructed by a set of support examples through an embedding function implemented by a deep network (Fig. 1A), can be optimized via end-to-end training. PTNs have displayed promising performances in benchmark

Z. Ji and X. Zou—Equal contribution.

© Springer Nature Switzerland AG 2018

L. Cheng et al. (Eds.): ICONIP 2018, LNCS 11303, pp. 603–611, 2018.

https://doi.org/10.1007/978-3-030-04182-3_53

datasets, and therefore have received large attention in the field. For instances, Oreshkin et al. explored different ways of defining the distance metric between data points [2], Sung et al. proposed an approach to learn a non-linear distance metric [3], Ren et al. augmented PTNs with the ability of using unlabeled examples to construct prototypes [4], and Liu et al. considered a transductive setting, which utilizes both the support and query sets to exploit the structure of class space across episodes [5].

The metric-learning idea in PTNs is very appealing to neural information processing, as the experimental data has indicated that neural systems categorize and recognize objects based on their semantic similarities [6]. The goal of this study is to investigate the potential link between PTNs and neural information processing. Motivated by the characteristics of neural systems, we extend the structure of PTNs in two aspects. Firstly, we know that a deep neural network mimics the layered architecture of the ventral visual pathway (Fig. 1B), and that in the visual system, object information are read-out at different layers, rather than only at the last layer as in a PTN. For instance, Matsumoto et al. [6] found that macaque monkeys can still assign stimuli to previously learned categories even after bilateral removal of the anterior inferior temporal cortex, and the latter is known to play an important role in object recognition. To mimic this property, we extend a PTN by considering that class prototypes are optimized at each layer of the network, which we call a hierarchical PTN (HPTN). Secondly, in a neural system, all computations are carried out via the dynamics of neural networks. The distance computation in a PTN is similar to the pattern overlap computation in the dynamics of an attractor network (Fig. 1C). It is valuable to explore whether a PTN can be realized by a biologically plausible attractor network. Overall, we hope this study will give us insight into understanding metric-learning in neural systems and shed light on developing advanced brain-inspired computational models.

The rest of the paper is organized as follows. Section 2 introduces the idea of HPTNs and their applications to two tasks on few-shot learning, Sect. 3 presents a link between PTNs and attractor neural networks, and Sect. 4 gives an overall conclusion.

2 Hierarchical Prototypical Networks

2.1 Prototypical Networks

To start, we briefly introduce the idea of PTNs. Following the fact that PTNs were originally proposed to solve the few-shot learning problem, we describe PTNs in the framework of few-shot learning. Specifically, we adopt the episodic learning paradigm proposed by Vinyals et al. [7], and call the condition that K examples for each of N classes are available in an episode to be N -way K -shot learning. Given the support set S , i.e., the training examples, a PTN computes the prototype of each class by the mean of the support data points belong to that class, which is written as,

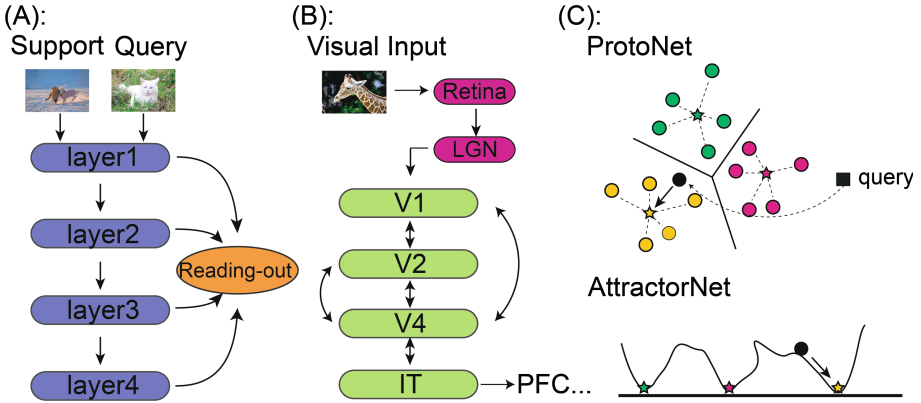


Fig. 1. (A). The architecture of a HPTN consists of four layers, each of which is formed by a Convolution-BatchNorm-ReLU-Pooling block. The classification probabilities at four layers are weighted properly to decide the final classification result. (B) The ventral visual pathway, along which object information are extracted and read-out layer by layer. (C) Upper panel: in a PTN, classification is done by computing the distances between the query data and prototypes; Lower panel: in an attractor neural network, classification is done through evolving the network state to one of its attractors.

$$c_m = \frac{1}{K} \sum_{(x_i, y_i) \in S_m} f_\phi(x_i), \quad (1)$$

where c_m denotes the prototype of class m and $S_m = \{(x_i, y_i)\}$, for $i = 1, \dots, K$, the corresponding support set. The function $f_\phi : R^D \rightarrow R^M$ represents the embedding function implemented by the deep network used, with ϕ the learnable parameters. When a new query data x is presented, the PTN calculates its probabilities belonging to N classes according to its distances to all prototypes, which is written as,

$$p_\phi(y = m|x) = \frac{\exp\{-d[f_\phi(x), c_m]\}}{\sum_n \exp\{-d[f_\phi(x), c_n]\}}, \quad (2)$$

where $d(\cdot)$ denotes the distance measure, and Euclidean distance is used in the present study. Through minimizing the negative log-probability $J(\phi) = -\log p_\phi(y = m|x)$ with respect to training examples, the PTN optimizes the network parameters ϕ , such that the network generates representations that are most suitable to classify data points based on their distance between each other.

2.2 Hierarchical Prototypical Networks

In contrast to a PTN, where prototypes are only explicitly optimized in the last layer of the network, in a HPTN, prototypes are defined at all layers and the network tries to minimize the weighted classification errors based on prototypes of all layers.

Suppose the network has L layers. Denote c_m^l the prototype of class m at layer l , which is calculated to be,

$$c_m^l = \frac{1}{K} \sum_{(x_i, y_i) \in S_m} f_{\phi^l}(x_i), \quad (3)$$

where $f_{\phi^l}(\cdot)$ represents the embedding function implemented by the first l layers of the network with ϕ^l is the corresponding parameters. Similarly, the class probabilities at layer l are calculated to be,

$$p_{\phi^l}(y = m|x) = \frac{\exp \{-\beta^l d[f_{\phi^l}(x), c_m^l]\}}{\sum_n \exp \{-\beta^l d[f_{\phi^l}(x), c_n^l]\}}. \quad (4)$$

Following the work of [2], we scale the distance metric by adding a learnable scalar β^l to each layer, which enables the HPTN to optimize the regime for each similarity metric $d[f_{\phi^l}(x), c_m^l]$.

The negative log-probability loss at layer l is given by $J_l(\phi^l) = -\log p_{\phi^l}(y = m|x)$. The overall loss function of the HPTN is defined to be

$$J(\phi) = \sum_{l=1}^L \alpha^{L-l} J_l(\phi^l), \quad (5)$$

where $0 \leq \alpha \leq 1$ is a discount factor. We assign larger weights to higher layers, since normally representations at higher layers better discriminate objects. When $\alpha = 0$, it returns to a PTN.

After training, the class probabilities for an unseen data are calculated by weighting the results from all layers, which are given by

$$p_{\phi}(y = m|x) = \frac{1}{Z} \sum_{l=1}^L \gamma^{L-l} p_{\phi^l}(y = m|x), \quad (6)$$

where Z is a normalization factor and γ the weighting factor. $\gamma = \alpha$ is used in the present study, but in practice γ can also be optimized via learning.

2.3 Experimental Results

In the experiments below, we choose the network architecture to be the same as that in [7], which consists of four stacked layers, with each layer containing a 3×3 convolution with 64 filters followed by batch normalization, a ReLU non-linearity, and 2×2 max-pooling. Prototypes are calculated at each layer after the ReLU operation.

Two datasets, Omniglot and miniImageNet, are used. The Omniglot dataset has 1623 classes from 50 alphabets [8]. There are 20 examples in each class and each of them was drawn by a different human subject. To reduce over-fitting, data augmentation was performed with rotations in multiples of 90 degrees. Following Vanyals et al. [7], all the gray scale images are resized to 28×28 .

We use 1200 characters plus rotations (4800 characters in total) for episodic training and the rest plus rotations for testing. The miniImageNet dataset was originally proposed by Vinyals et al. [7], which is derived from the ILSVRC-12 dataset [9]. It consists of 100 classes and each class contains 600 colored images of size 84×84 . In our experiments, we use the splits introduced by Ravi and Larochelle [10], which has a different set of 100 classes including 64 for training, 16 for validation, and 20 for test, and the validation dataset is only used for early-stopping. We find that it takes long time to calculate distances in shallow layers due to the large dimensionality of shallow features, we therefore scale down the original 84×84 images to 64×64 , but the performance of PTNs is retained. Similar to [1], we set 60-way episodes for 1-shot and 5-shot classifications in the Omniglot task, and 30-way episodes for 1-shot classification and 20-way episodes for 5-shot classification in the miniImageNet task.

Table 1 presents the experiment results for the Omniglot task. Since the authors didn't release the detailed configuration, we could not reproduce the results in [1]. Therefore we implement PTNs by ourselves using the same hyperparameters as in [1] but with more training epochs. We see that the overall performance of the HPTN is comparable to that of the PTN in [1] but better than our implementation of the PTN. The results for the miniImageNet are similar, see Table 2.

Table 1. Classification accuracies on Omniglot. In our implementation of PTN and HPTN, we initialize all the parameters with the same seed. Training epochs is set to be 200, episodic training classes is 60, learning rate is initialized to be 10^{-3} , and we cut the learning rate in half every 4000 episodes. No regularization was used other than batch normalization.

Model	5-way		20-way	
	1-shot	5-shot	1-shot	5-shot
Neural Statistician [11]	98.1	99.5	93.2	98.1
Matching Networks [7]	98.1	98.9	93.8	98.5
PTN [1]	98.8	99.7	96.0	98.9
PTN (Ours)	98.7	99.6	95.4	98.6
HPTN	98.9	99.7	95.6	98.8

Interestingly, we observe that the classification accuracies in the intermediate layers of the HPTN are significantly improved compared to that of the PTN (note that in the PTN, although we did not include classification errors of the intermediate layers in the loss function, we can still calculate their classification performances by computing the prototypes using the support set accordingly). As shown in Fig. 2, for the Omniglot task, the improvements of the HPTN are around 5% in layer 2 and 3% in layer 3 compared to the PTN (Fig. 2A); for the miniImageNet task, the improvements are around 3% in layer 2 and 5% in layer 3 (Fig. 2B). These improvements are intuitively understandable, since the HPTN

Table 2. Classification accuracies on miniImageNet. Parameters are initialized with the same seed. Training epochs are set to be 200. We use 30-way episodes for one-shot classification and 20-way episodes for 5-shot classification. Learning rate is initialized to be 10^{-3} , and we cut the learning rate in half every 4000 episodes. No regularization was used other than batch normalization.

Model	5-way	
	1-shot	5-shot
Matching Networks [7]	46.6	60.0
Meta-Learner LSTM [10]	43.4	60.6
PTN [1]	49.4	68.2
PTN (Ours)	48.6	65.5
HPTN	49.7	67.3

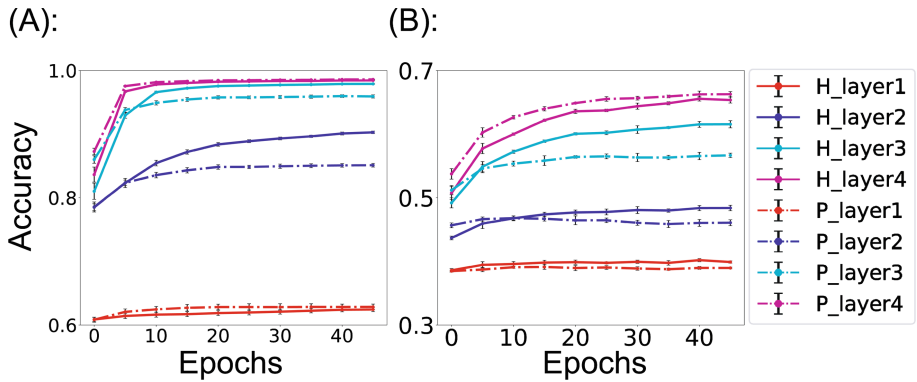


Fig. 2. Comparing classification accuracies at different layers of a HPTN and a PTN, denoted by solid and dashed lines, respectively. Different layers are colour-coded. The results are obtained by averaging over 1000 test episodes and reported with 95% confidence interval. A. The Omniglot dataset, 20-way 5-shot learning. B. The miniImageNet dataset, 5-way 5-shot learning. (Color figure online)

is trained to minimize also the classification errors in the intermediate layers of the network (Eq. (5)). This property is appealing to a neural system, as it means that the HPTN not only achieves an overall good classification performance, but also generates a set of good representations of increasing complexity along the network hierarchy. These hierarchical representations reflect that object features are extracted from simple to complex along the network hierarchy, and that the object information can be read-out from different layers depending on the requirement, achieving the so-called rough-to-fine object recognition.

3 Linking to Attractor Neural Networks

The appealing properties of PTNs/HPTNs are that, on one hand, they classify objects based on the distances/similarities between objects (metric-learning), and on the other hand, they provide an efficient way to optimize the representations of objects via end-to-end supervised training. Notably, the distance computation in a PTN/HPTN (in particular, when the Euclidean distance is used) is analogy to the pattern overlap computation in the attractor dynamics of a Hopfield network, and the latter is known to mimic information retrieval in neural systems [12]. Therefore, we explore whether the classification operation via distance computation in a PTN/HPTN can be replaced by the dynamics of an attractor network. For illustration, we only consider that the last step of classification by distance in a PTN is replaced by the dynamics of a Hopfield network, but extension to multiple layers in a HPTN is straightforward.

To construct a Hopfield model, we binarize the activities in the last layer of a PTN (± 1) and define the prototypes of all classes as the patterns the Hopfield network needs to memorize, which are denoted as μ_i , for $i = 1, \dots, N$. We adopt two ways to construct the neuronal connections $\mathbf{W}$: one uses the conventional Hebb rule, which gives rise to $\mathbf{W} = \sum_i^N \mu_i^T \mu_i / N$, and the other uses the orthogonal Hebb rule, which orthogonalizes the memory patterns before applying the Hebb rule (for detail, see [13]). It is known that only the orthogonal Hebb rule ensures that the Hopfield model holds correlated memory patterns as its attractors ([14]). Once the Hopfield network is established, given a query pattern, the network evolves to an attractor (corresponding to a prototype) automatically, which outputs the class label of the input (see illustration in Fig. 1C).

Table 3 summarizes the experimental results, which show that: (1) when the representations are binarized (no attractor dynamics is applied yet), the performance of the PTN is slightly decreased compared to the case of using continuous representations; (2) when the Hopfield model with the conventional Hebb rule is applied, the performance of the network is degraded dramatically, in particular, in the difficult 20-way learning case; (3) when the Hopfield model with the orthogonal Hebb rule is applied, the performance of the network is still

Table 3. Classification accuracies of PTNs and a Hopfield model. Features are binarized according to mean values. The embedding network is trained using the Omniglot dataset. The hyper-parameters are the same as in Sect. 2.3.

Model	5-way		20-way	
	1-shot	5-shot	1-shot	5-shot
PTN	98.6	99.6	94.9	98.5
Binarized PTN	96.5	98.6	87.4	94.7
Hopfield with Hebb	89.1	92.9	25.9	27.3
Hopfield with Ortho_Hebb	94.4	97.9	78.2	90.6

decreased compared to the PTN, but the gap is not huge. Overall, our study indicates that a prototypical network can be approximately implemented by a biologically plausible attractor network.

4 Conclusions

In the present study, motivated by the characteristics of neural systems, we have extended PTNs in two aspects. First, we developed HPTNs to optimize representations for prototypes at all layers. Applied to benchmark datasets, the overall performances of HPTNs are comparable to, or slightly better than, that of PTNs. However, HPTNs generate much better representations for prototypes at the intermediate layers of the network. Secondly, we explored a potential link between PTNs and attractor neural networks by demonstrating that the classification operation via distance in PTNs can be realized approximately by the attracting dynamics of the Hopfield model. We hope that this study gives us insight into understanding metric-learning in neural systems (i.e., processing information based on semantic similarity) and sheds light on developing advanced brain-inspired computational models.

Acknowledgement. This work was supported by National Key Basic Research Program of China (2014CB846101), BMSTC (Beijing municipal science and technology commission) under grant No: Z161100000216143(SW), Z171100000117007(DHW & YYM); the National Natural Science Foundation of China (No: 31771146, 11734004, YYM), Beijing Nova Program (No: Z181100006218118, YYM).

References

1. Snell, J., Swersky, K., Zemel, R.: Prototypical networks for few-shot learning. In: *Advances in Neural Information Processing Systems*, pp. 4080–4090 (2017)
2. Oreshkin, B.N., Lacoste, A., Rodriguez, P.: TADAM: task dependent adaptive metric for improved few-shot learning. arXiv preprint [arXiv:1805.10123](https://arxiv.org/abs/1805.10123) (2018)
3. Sung, F., Yang, Y., Zhang, L., Xiang, T., Torr, P.H., Hospedales, T.M.: Learning to compare: relation network for few-shot learning. arXiv preprint [arXiv:1711.06025](https://arxiv.org/abs/1711.06025) (2017)
4. Ren, M., et al.: Meta-learning for semi-supervised few-shot classification. arXiv preprint [arXiv:1803.00676](https://arxiv.org/abs/1803.00676) (2018)
5. Liu, Y., Lee, J., Park, M., Kim, S., Yang, Y.: Transductive propagation network for few-shot learning. arXiv preprint [arXiv:1805.10002](https://arxiv.org/abs/1805.10002) (2018)
6. Matsumoto, N., Eldridge, M.A., Saunders, R.C., Reoli, R., Richmond, B.J.: Mild perceptual categorization deficits follow bilateral removal of anterior inferior temporal cortex in rhesus monkeys. *J. Neurosci.* **36**(1), 43–53 (2016)
7. Vinyals, O., Blundell, C., Lillicrap, T., Wierstra, D.: Matching networks for one shot learning. In: *Advances in Neural Information Processing Systems*, pp. 3630–3638 (2016)
8. Lake, B.M., Salakhutdinov, R., Tenenbaum, J.B.: Human-level concept learning through probabilistic program induction. *Science* **350**(6266), 1332–1338 (2015)

9. Russakovsky, et al.: ImageNet large scale visual recognition challenge. *Int. J. Comput. Vis.* **115**(3), 211–252 (2015)
10. Ravi, S., Larochelle, H.: Optimization as a model for few-shot learning (2016)
11. Edwards, H., Storkey, A.: Towards a neural statistician. *arXiv preprint [arXiv:1606.02185](https://arxiv.org/abs/1606.02185)* (2016)
12. Hopfield, J.J.: Neural networks and physical systems with emergent collective computational abilities. *Proc. Natl. Acad. Sci.* **79**(8), 2554–2558 (1982)
13. Srivastava, V., Sampaath, S., Parker, D.J.: Overcoming catastrophic interference in connectionist networks using Gram-Schmidt orthogonalization. *PloS One* **9**(9), e105619 (2014)
14. Zou, X., Ji, Z., Liu, X., Mi, Y., Wong, K.M., Wu, S.: Learning a continuous attractor neural network from real images. In: Liu, D., Xie, S., Li, Y., Zhao, D., El-Alfy, E.S. (eds.) *ICONIP 2017. LNCS*, vol. 10637, pp. 622–631. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-70093-9_66